Verification of Templated Code in C++

Gregory Malecha
gregory@skylabs-ai.com
Skylabs AI
Massachusetts, USA

David Swasey*
pswasey@riversideresearch.org
Riverside Research
Massachusetts, USA

Simon Hudon
simon@skylabs-ai.com
SkyLabs AI
Massachusetts, USA

Abstract

C++ templates are used pervasively and a major blocker for applying verification to real code bases. We present an approach to verifying templated code that avoids enlarging the trusted computing base of the C++ semantics. Our approach uses a shallowly embedded logic that enables reasoning about unresolved symbols as hygienic macros. We have applied our approach to verify several small function templates and used their proofs to derive proofs for concrete instantiations of these functions.

CCS Concepts: • **Theory of computation → Program verification**.

Keywords: C++, templates, macros, separation logic

1 Introduction

While C++ started out as "C with classes", modern usage of it would better describe it as "C with templates". C++ templates provide a way to express polymorphic functions and data types and are used pervasively throughout the standard library in abstractions ranging from std :: optional<T> to std :: vector<T>. Parametric and ad hoc polymorphism, which templates provide, are a cornerstone of functional programming languages; however, the design of templates within C++ is unique in important ways. In particular, templated code is only type-checked after instantiation which means that unresolved nodes act more like hygienic macros than typed function calls. While this limits eager error reporting, it integrates well with C++'s support for overloading.

We aim to verify templated code once and use the proof to verify its many instantiations *without enlarging the size of the C++ semantics*. We achieve this using deeply defined substitutions and a shallowly defined program logic. Figure 1 gives an example of a simple inc_twice template that increments its argument twice and its soundness theorem. The soundness theorem proves the correctness of the body of the code for any substitution (σ) where the pre-increment operator satisfies a certain specification. These choices lead to simple lemmas that express the correctness of templated code by quantifying over substitutions constrained by reasoning principles for unresolved constructs that occur within the implementation. Using the soundness theorem, we can verify any instantiation of inc_twice that satisfies the constraints fulfilling our goals as long as inc_twice_ok is axiom free.

We present the three key aspects of our approach:

```
template<typename T>
void int_twice(T& x) { ++(++x); }
```

```
Lemma inc_twice_ok : ∀ tu σ,
  template_tu ⊆ tu → (* source code AST *)
  (* specification of unresolved operators *)
  unresolved_spec "operator++(T&)" "T&"
    (λ E Q. E (λ p. ∃ v, p↦R v ∗ (p↦R (v+1) −∗ Q p))) →
  (* final specification of the function *)
  template_spec (subst σ ...inc_twice_body...)
    (\ arg{r} "x" (Vref r)
     \pre{v} r ↦ R v
     \post   r ↦ R (v + 2)).
Proof. ... Qed.
```

Figure 1. A simple templated C++ function and its soundness theorem.

1. Formalizing templated syntax (§2) and the semantics of instantiation (§3),
2. Lifting the program logic to templated syntax (§4), and
3. Specifying unresolved symbols using contexts (§4.1).

We will focus on function templates, but the techniques also work for templated data structures.

Background: The BRiCk Program Logic. BRiCk [1] provides an axiomatic semantics for the runtime features of C++, *e.g.* operators, function calls, etc, on top of a "fully elaborated" AST in which overload resolution and type checking have already occurred. Operating at this level enables BRiCk to avoid the complexities of the "static semantics" of the language and focus on the runtime semantics.

2 Template Syntax

We extend the BRiCk syntax with a representation of unresolved nodes, which represent multi-holed contexts. Due to overloading in C++, the names of these holes are dependent on the types of the arguments. For example, the implementation of inc_twice in Figure 1 has *2 distinct* unresolved operators. The inner pre-increment operator has the name Uunop Rpreinc "*T*&" where "*T*&" is the type of the argument.

The second pre-increment operator is different because the type of the argument is not (necessarily) *T*&, but is rather the type of the preceding pre-increment. In C++, this type can be written decltype((++x)), but to abstract this from the x used to fill the hole, we use the syntax Tunop Rpreinc "*T*&" to name this type. This leads to the second pre-increment operator having the name: Uunop Rpreinc (Tunop Rpreinc "*T*&").

*Work carried out while at BlueRock Security, Inc.

The implicit introduction of new types in C++ differs from languages such as Haskell and Rocq where all of these types occur explicitly in the signature.

3 Template Instantiation

Template instantiation replaces unresolved nodes with their implementation using C++'s overload resolution mechanism. Two difficulties arise with formalizing overload resolution.

Substitution Witnesses. Overload resolution combines logic programming and constexpr evaluation (to support features such as requires) and requires a complete view of the program, i.e. it is not stable as more definitions are added to the program. These both lead to difficulties in formalizing it which we can side-step by using a concrete representation of substitutions (which we refer to using σ) rather than implementing substitution using the program context. For example, a concrete substitution contains a finite map from unary operator names to their instantiations (currently represented as Expr → Expr). To ease applying the soundness theorem from Figure 1, we have implemented a unification function that build a concrete substitution from the templated code and its instantiation. Crucially, this operation does not need to be trusted since applying the lemma will implicitly check the substituted term matches the target term.

Bottom-up Instantiation. Because instantiation occurs bottom up in C++, resolution uses the instantiated types of the arguments rather than the templated types that occur in names (§2). Concretely when using `T = int`, the resolution of the outer pre-increment in inc_twice needs to use `Uunop Rpreinc "int&"` where **int**& is the result of instantiating `Tunop Rpreinc "T&"`. Because of this, the keys in substitutions are instantiated types and the lookup operation needs to invoke substitution[1].

4 A Program Logic for Templates

Using substitution and the weakest precondition semantics from BRiCk, we *define* a weakest precondition for templated terms. Here we show the one for statements for simplicity.

```
Definition wp_mstmt σ tu ρ Ms Q :=
  ∀ s, [| subst σ Ms = mret s |] -* wp_mstmt tu ρ s Q.
```

The use of ∀ and −∗ here is crucial and allows the prover of a weakest pre-condition to assume that instantiation succeeds rather than being obligated to prove that it does.

This phase splitting enables us lift the rules about non-templated constructs to the templated language in a straightforward manner. For sequential composition,

```
∀ σ tu ρ s1 s2,
  wp_mstmt σ tu ρ s1 (wp_mstmt σ tu ρ s2 Q)
  ⊢ wp_mstmt σ tu ρ (s1; s2) Q.
```

[1]We currently break the circularity using a second level of substitution.

Proving this turns out to be easy since the conclusion contains the fact that the substitution succeeds on `s1; s2` which allows us to discharge the pre-conditions to the wp_mstmt's in the premise. If we had used ∃ and ∗ rather than ∀ and −∗ in the definition of wp_mstmt, we would be stuck proving that substitution succeeds on s2 since the fact is only available in the normal termination continuation for s1.

4.1 Context Specifications

What remains is to establish the weakest pre-condition of unresolved nodes. Unresolved nodes expand to multi-holed contexts rather than function calls due to implicit coercions and differs from the semantics of other languages. For example,

```
template<typename T>
void add(T& l, T& r) { l + r; }
```

This function has one unresolved node for the plus operator which expands to the following when `T = int`.

```
CTX x y :=
  Ebinop Bplus (Eicast Cl2r x) (Eicast Cl2r y)
```

Note that each argument is wrapped in an implicit left-to-right cast. Schematically, every context is of the form: $F\ (C_0\ x_0) \ldots (C_n\ x_n)$ where each C_i represents a chain of implicit casts. Because C++ sequences argument evaluation, which is important in the presence of exceptions, the evaluation of portions of the context will interleave with the evaluation of the context's arguments.

To capture the potential for interleaved execution, we specify contexts as higher-order predicate transformers. A concrete example of specification for a unary pre-increment operator is shown in Figure 1 and repeated here:

```
unresolved_spec (UnOp Rpreinc "T&") "T&"
  (λ E Q. E (λ p. ∃ v, p↦R v ∗ (p↦R (v+1) -∗ Q p)))
```

Here, `UnOp Rpreinc "$T&"` is the name of the unresolved symbol, and `"$T&"` is the return type, i.e. `Tunop Rpreinc "T&"` $\cong_\sigma$ `"T&"`. The final argument is the specification of the context. E is the predicate transformer for the argument, and Q is the continuation. The body runs the argument with a continuation that increments the model of the R predicate at p, where R is the ownership of the C++ type T.

5 Applications

We have used this approach to verify several simple function templates such as the inc_twice example as well as the copy-assignment operator for a std :: pair-like class. Our experience to date is that the theory of the lifted weakest pre-conditions makes the verification mostly straightforward modulo the complexity of reasoning about context-specifications, which can be quite tricky. In practice, it seems like disciplined uses of implicit casts, namely that they do not have non-local side-effects, should keep the reasoning tractable.

References

[1] 2025. The BRiCk Project. https://github.com/SkylabsAI/BRiCk.